CS 575 Lab Notebook

Lab 3

Name: Jarren Calizo
OdinID: calizo
Date: January 24, 2026

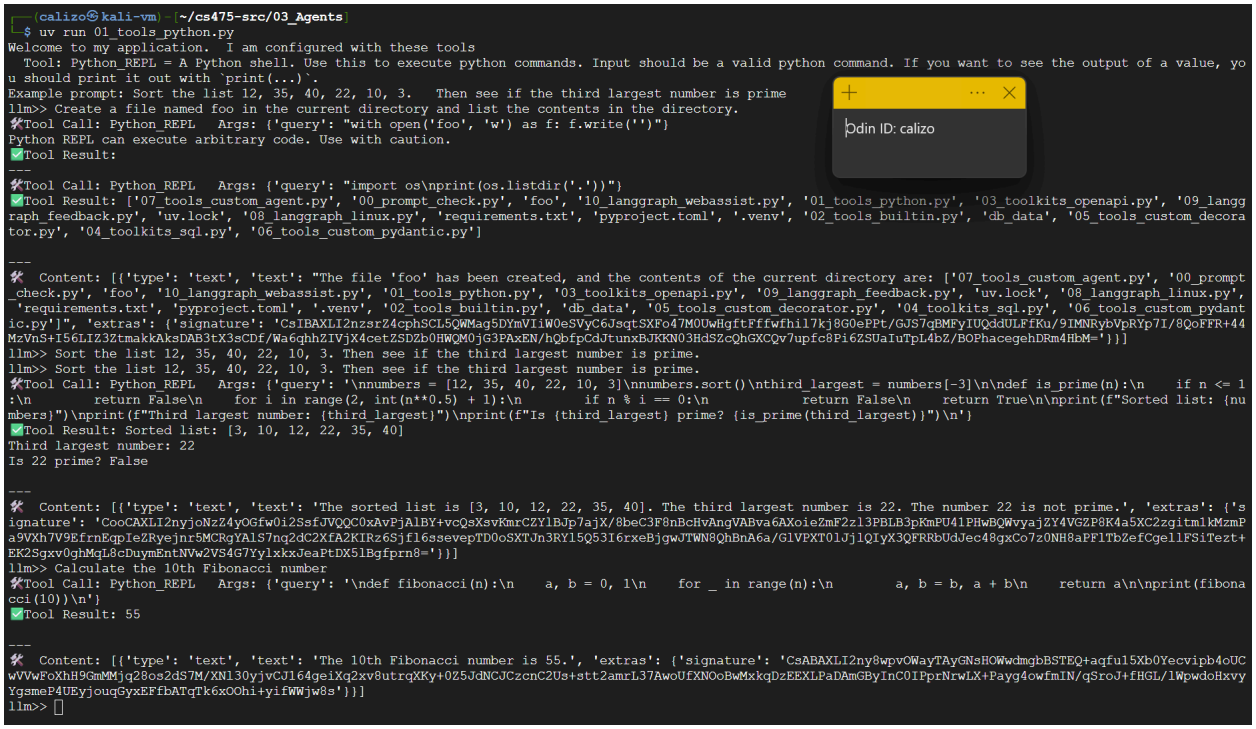
Table of Contents

Lab 3.1 LangChain Agents 2
Step 3. Tools 2
1. Take screenshots of your results that includes your OdinID 2
2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result 2
Step 4. Built-in tools 4
1. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result 4
Step 6. Toolkits (Open API) 2 7
1. Create a table for your lab notebook showing the trace of actions used to handle each prompt and the final result 7
Step 8. Toolkits (SQL) 2 10
1. Take screenshots of your results that includes your OdinID 10
2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result 12
3. Take screenshots of your results that includes your OdinID 15
4. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result 16
Step 10. Custom tools via @tool (2) 18
1. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result 18
2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the result of the injection attempt 20
Step 11. Validating tool input 22
1. Take a screenshot of the result that includes your OdinID 22
Step 12. Custom agent 22
1. Take screenshots of your results that includes your OdinID 22
2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result 22
Step 14. LangSmith (2) 23
1. Take a screenshot of the execution trace on LangSmith that includes your OdinID 24
Step 16. LangGraph introduction (2) 25
1. Take a screenshot of the result that includes your OdinID 25
Step 19. LangGraph human-in-the-loop tool calling (3) 25
1. Take a screenshot of the results that includes your OdinId 25
Step 21. LangGraph web assistant (2) 26
1. Take a screenshot of the application graph that includes your OdinId 26
2. Take a screenshot of the query and output for each that includes your OdinId 27

Lab 3.1 LangChain Agents

Step 3. Tools

1. Take screenshots of your results that includes your OdinID



2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result

Prompt	Thought	Action	Action Input	Observation	Final Answer
Create a file named foo in the current directory and list the contents	Create the file first	Python _REPL	with open('foo','w') as f: f.write('')	File created (no printed output)	
Create a file named foo in the current directory and list the contents	List directory contents	Python _REPL	import os print(os.listdir('.'))	Directory listing includes foo	The file 'foo' has been created, and the contents of the current directory are: ['07_tools_custom_a

					gent.py', '00_prompt_check.py', 'foo', '10_langgraph_webassist.py', '01_tools_python.py', '03_toolkits_openapi.py', '09_langgraph_feedback.py', 'uv.lock', '08_langgraph_linux.py', 'requirements.txt', 'pyproject.toml', '.venv', '02_tools_builtin.py', 'db_data', '05_tools_custom_decorator.py', '04_toolkits_sql.py', '06_tools_custom_pydantic.py']
Sort the list 12, 35, 40, 22, 10, 3. Then see if the third largest number is prime	Sort + compute 3rd largest + prime check	Python_REPL	numbers=[12,35,40,22,10,3] numbers.sort() third_largest=numbers[-3] is_prime(third_largest) print(f"Sorted list: {numbers}") print(f"Third largest number: {third_largest}") print(f"Is {third_largest} prime? {is_prime(third_largest)}")	Sorted list: [3, 10, 12, 22, 35, 40] Third largest: 22 Prime?: False	The sorted list is [3, 10, 12, 22, 35, 40]. The third largest number is 22. The number 22 is not prime.
Calculate the 10th	Compute Fibonacci	Python_REPL	def fibonacci(n):	Output: 55	The 10th Fibonacci number is 55

Fibonacci number	iteratively		a, b = 0, 1 for _ in range(n): a, b = b, a + b return a print(fibonacci(10))		

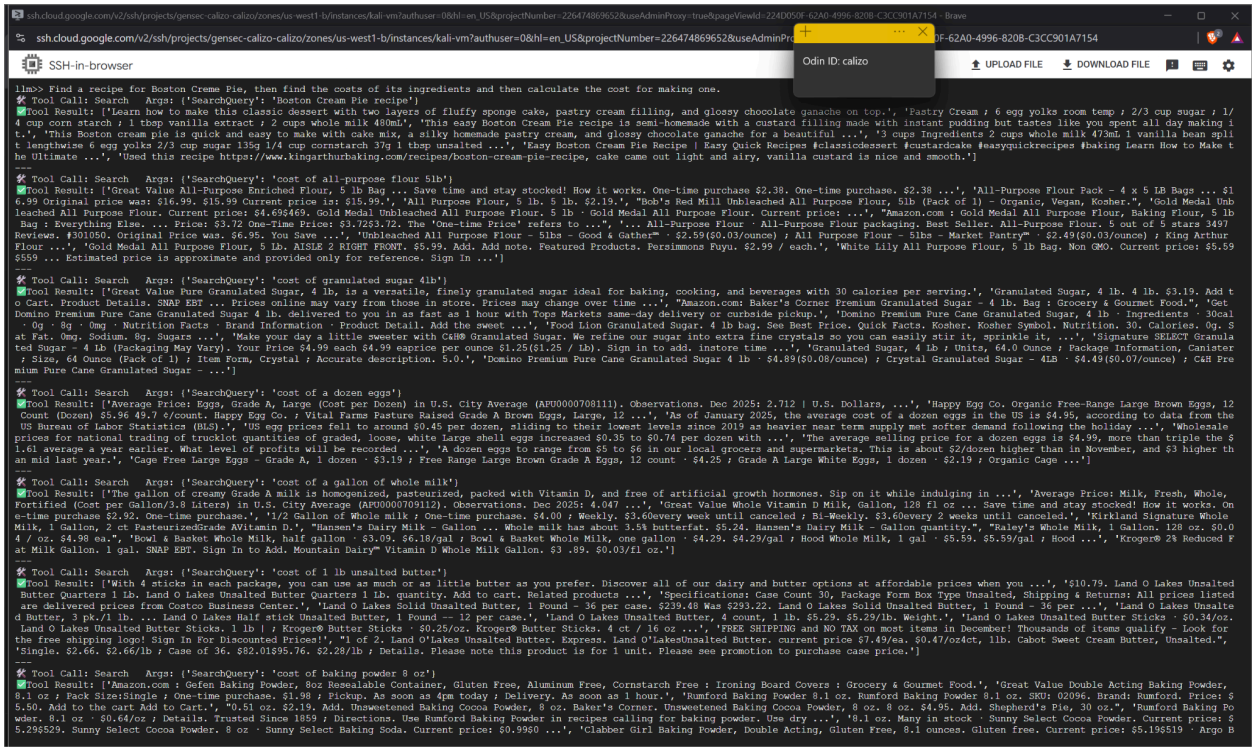
Step 4. Built-in tools

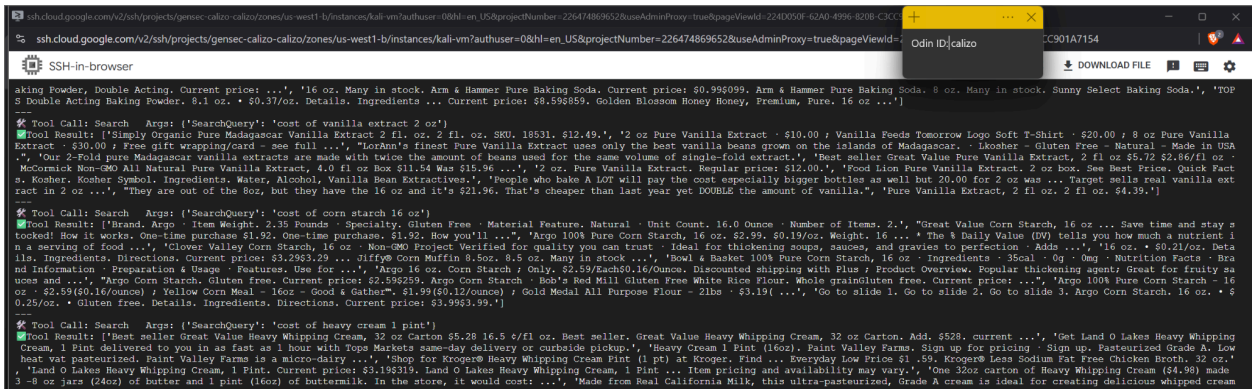
1. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result

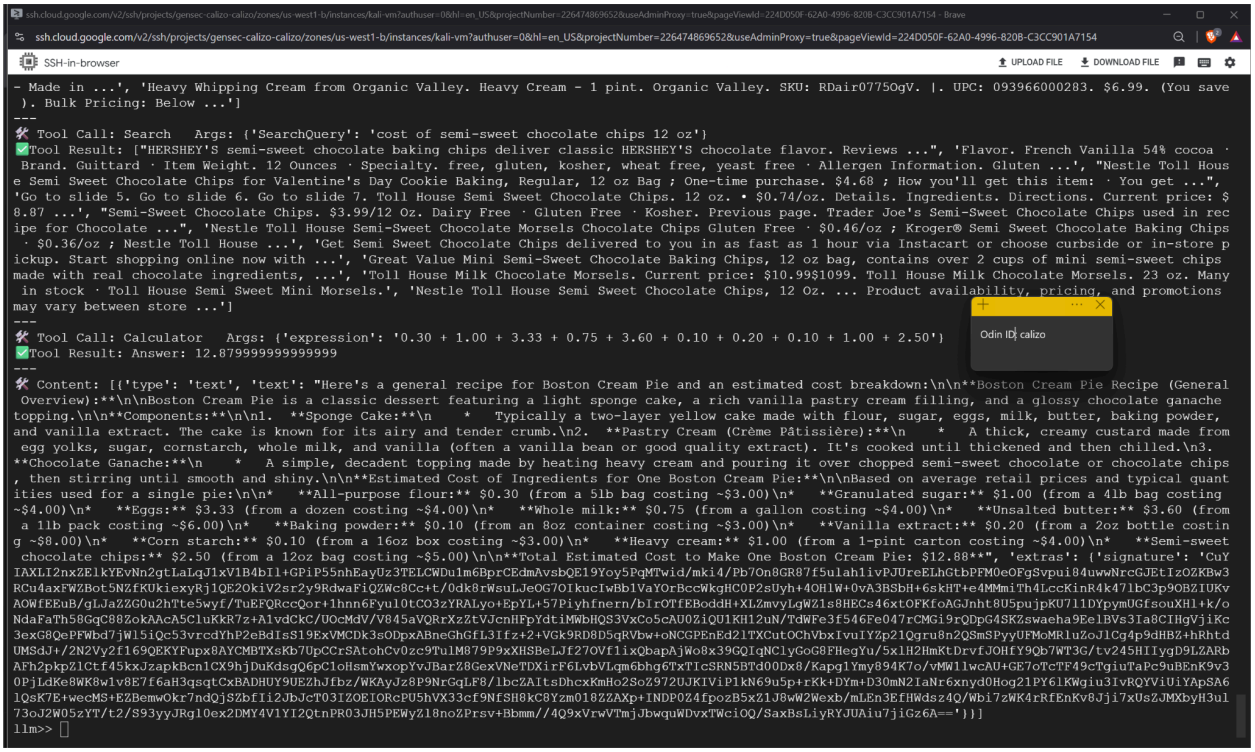
Prompt	Thought	Action	Action Input	Observation	Final Answer
Find a recipe for Boston Creme Pie, then find the costs of its ingredients and then calculate the cost for making one.	Get a baseline recipe first	Search	SearchQuery: "Boston Cream Pie recipe"	Returns a Boston Cream Pie recipe overview + ingredients	Recipe source text retrieved
"	Price-check main cake ingredient	Search	SearchQuery: "cost of all-purpose flour 5lb"	Returns product listings/prices for 5 lb flour	Flour price info retrieved
"	Price-check sugar	Search	SearchQuery: "cost of granulated sugar 4lb"	Returns product listings/prices for 4 lb sugar	Sugar price info retrieved
"	Price-check eggs	Search	SearchQuery: "cost of a dozen eggs"	Returns product listings/prices for a dozen eggs	Egg price info retrieved

"	Price-check milk	Search	SearchQuery: "cost of a gallon of whole milk"	Returns product listings/prices for a gallon of whole milk	Milk price info retrieved
"	Price-check butter	Search	SearchQuery: "cost of 1 lb unsalted butter"	Returns product listings/prices for 1 lb butter	Butter price info retrieved
"	Price-check baking powder	Search	SearchQuery: "cost of baking powder 8 oz"	Returns product listings/prices for ~8 oz baking powder	Baking powder price info retrieved
"	Price-check vanilla extract	Search	SearchQuery: "cost of vanilla extract 2 oz"	Returns product listings/prices for ~2 oz vanilla extract	Vanilla extract price info retrieved
"	Price-check cornstarch	Search	SearchQuery: "cost of corn starch 16 oz"	Returns product listings/prices for ~16 oz cornstarch	Cornstarch price info retrieved
"	Price-check heavy cream	Search	SearchQuery: "cost of heavy cream pint"	Returns product listings/prices for ~1 pint heavy cream	Heavy cream price info retrieved
"	Price-check chocolate chips	Search	SearchQuery: "cost of semi-sweet chocolate chips 12 oz"	Returns product listings/prices for ~12 oz chips	Chocolate chips price info retrieved
"	Sum estimated per-recipe ingredient costs	Calculator	expression: "0.30 + 1.00 + 3.33 + 0.75 + 3.60 + 0.10 + 0.20 + 0.10 + 1.00 + 2.50"	Answer: 12.879999999999999	Total estimated cost: ~$12.88
"	Present recipe + cost breakdown clearly	N/A	N/A	Final narrative response generated	Output included: recipe overview + estimated ingredient costs (flour $0.30, sugar $1.00, eggs $3.33, milk $0.75,

					butter $3.60, baking powder $0.10, vanilla $0.20, cornstarch $0.10, heavy cream $1.00, chocolate chips $2.50) and total $12.88







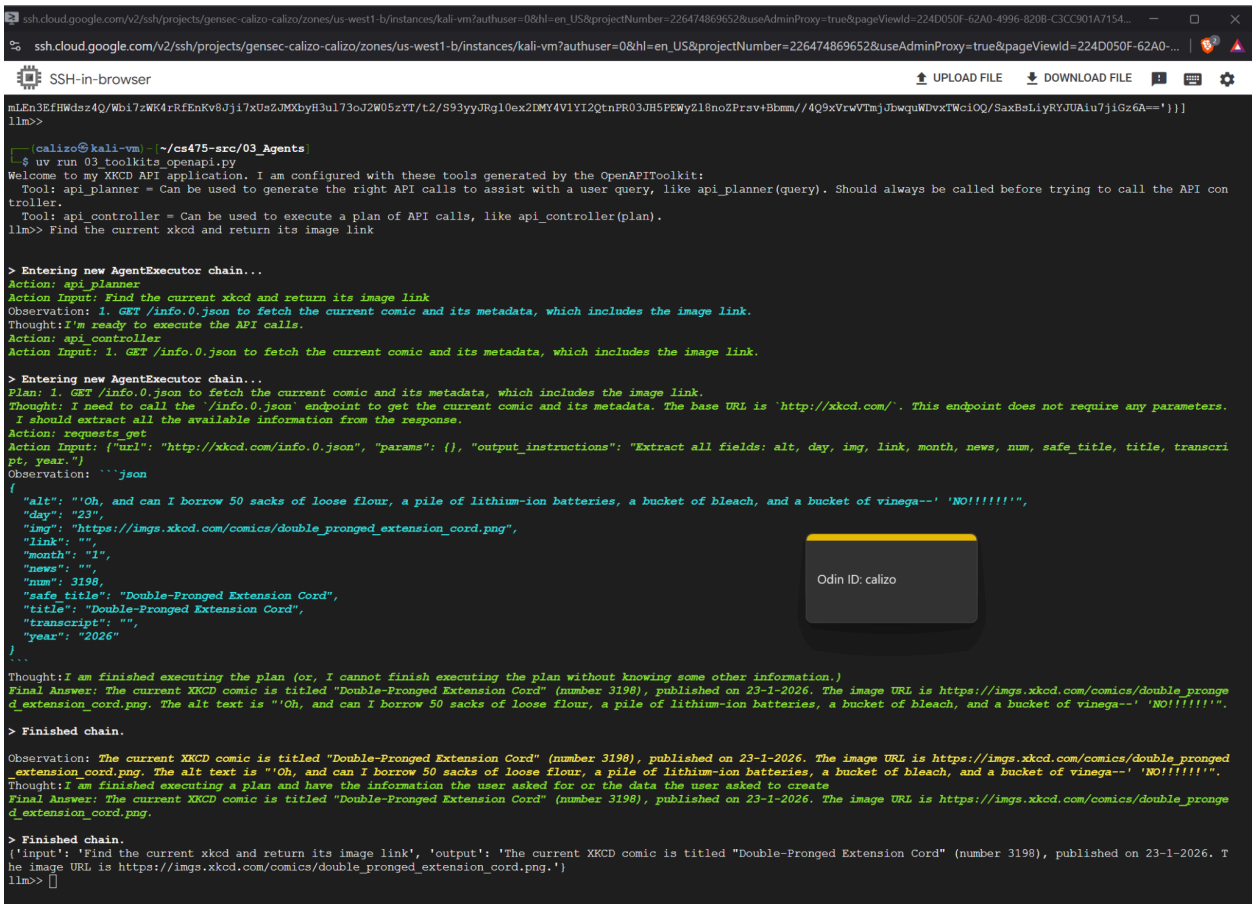
Step 6. Toolkits (Open API) 2

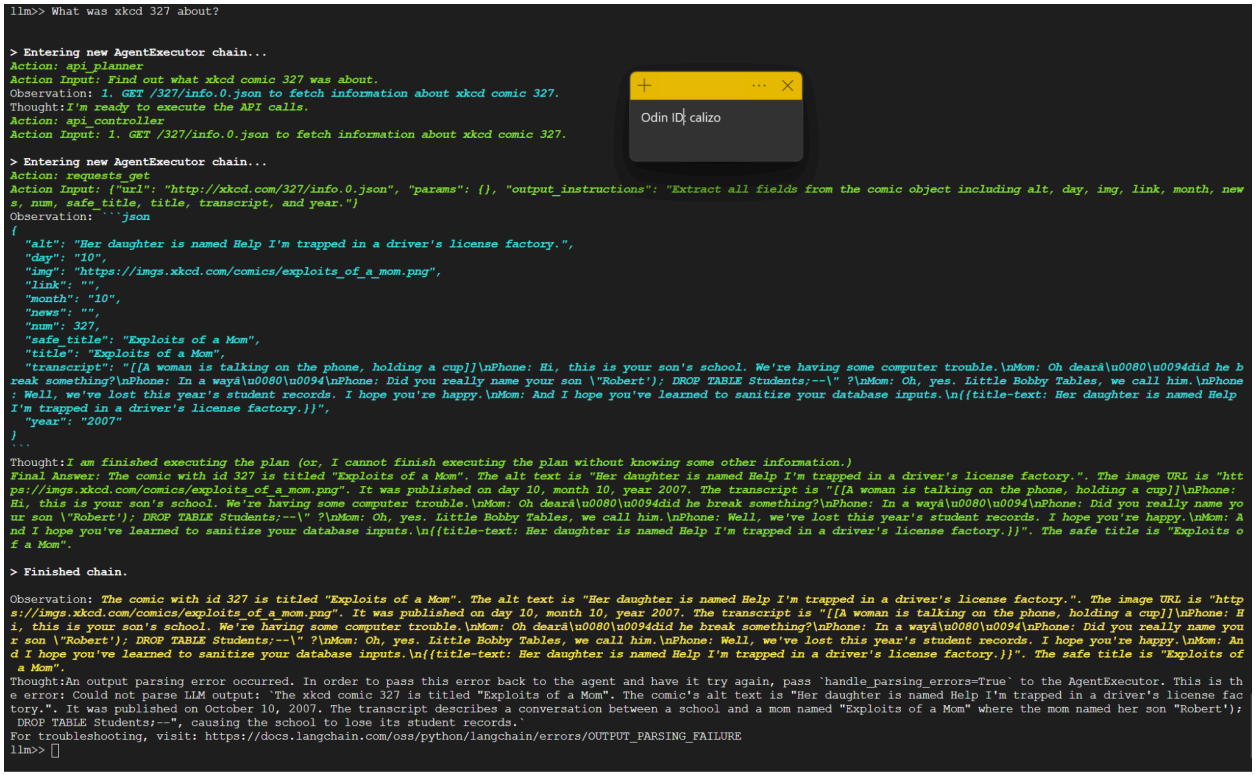
1. Create a table for your lab notebook showing the trace of actions used to handle each prompt and the final result

Prompt	Thought	Action	Action Input	Observation	Final Answer
Find the current xkcd and return its image link	Plan the right endpoint(s) to call	api_planner	Find the current xkcd and return its image link	Plan: GET /info.0.json (current comic metadata includes image link)	
Find the current xkcd and return its image link	Execute the plan	api_controller	GET /info.0.json	Controller proceeds to fetch from xkcd base URL	
Find the current xkcd and return its	Fetch JSON and extract fields	requests_get	url: http://xkcd.com/info.0.j	JSON returned: num 3198,	Current xkcd is "Double-Pron

image link			son output_instructions: extract fields (alt, day, img, link, month, news, num, safe_title, title, transcript, year)	title/safe_title "Double-Pronged Extension Cord", date 2026-01-23, img https://imgs.xkcd.com/comics/double_pronged_extension_cord.png, alt text about borrowing flour/batteries/bleach/vinegar	ged Extension Cord" (#3198), published 2026-01-23. Image URL: https://imgs.xkcd.com/comics/double_pronged_extension_cord.png
What was xkcd 327 about?	Plan the comic-specific endpoint	api_planner	Find out what xkcd comic 327 was about.	Plan: GET /327/info.0.json	
What was xkcd 327 about?	Execute the plan	api_controller	GET /327/info.0.json	Controller proceeds to fetch comic 327 metadata	
What was xkcd 327 about?	Fetch JSON and extract fields	requests_get	url: http://xkcd.com/327/info.0.json output_instructions: extract fields (alt, day, img, link, month, news, num, safe_title, title, transcript, year)	JSON returned: num 327, title/safe_title "Exploits of a Mom", date 2007-10-10, img https://imgs.xkcd.com/comics/exploits_of_a_mom.png, transcript about SQL injection: "Robert'); DROP TABLE Students;--"	xkcd 327 ("Exploits of a Mom") is a joke about SQL injection via a kid's name causing a database command to execute; published 2007-10-10. Image URL: https://imgs.xkcd.com/comics/exploits_of_a_mom.png. (After printing the

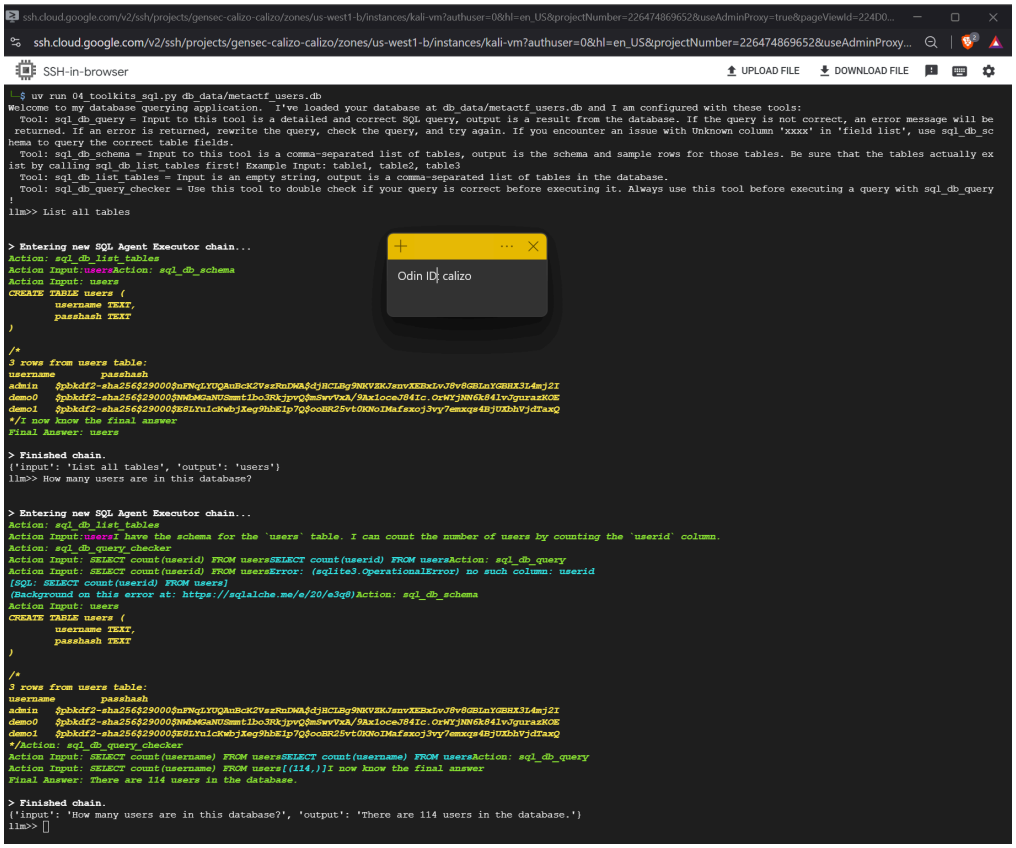
				and "sanitize your database inputs"	answer, the run shows an OUTPUT_PARSING_FAILURE error from the agent output parser.)





Step 8. Toolkits (SQL) 2

1. Take screenshots of your results that includes your OdinID




```
└─$ uv run 04_toolkits_sql.py db_data/metactf_users.db
Welcome to my database querying application.  I've loaded your database at db_data/metactf_users.db and I am configured with these tools:
  Tool: sql_db_query = Input to this tool is a detailed and correct SQL query, output is a result from the database. If the query is not correct, an error message will be returned. If an error is returned, rewrite the query, check the query, and try again. If you encounter an issue with Unknown column 'xxxx' in 'field list', use sql_db_schema to query the correct table fields.
  Tool: sql_db_schema = Input to this tool is a comma-separated list of tables, output is the schema and sample rows for those tables. Be sure that the tables actually exist by calling sql_db_list_tables first! Example Input: table1, table2, table3
  Tool: sql_db_list_tables = Input is an empty string, output is a comma-separated list of tables in the database.
  Tool: sql_db_query_checker = Use this tool to double check if your query is correct before executing it. Always use this tool before executing a query with sql_db_query!
llm>> List all tables


> Entering new SQL Agent Executor chain...
Action: sql_db_list_tables
Action Input:usersAction: sql_db_schema
Action Input: users
CREATE TABLE users (
        username TEXT,
        passhash TEXT
)

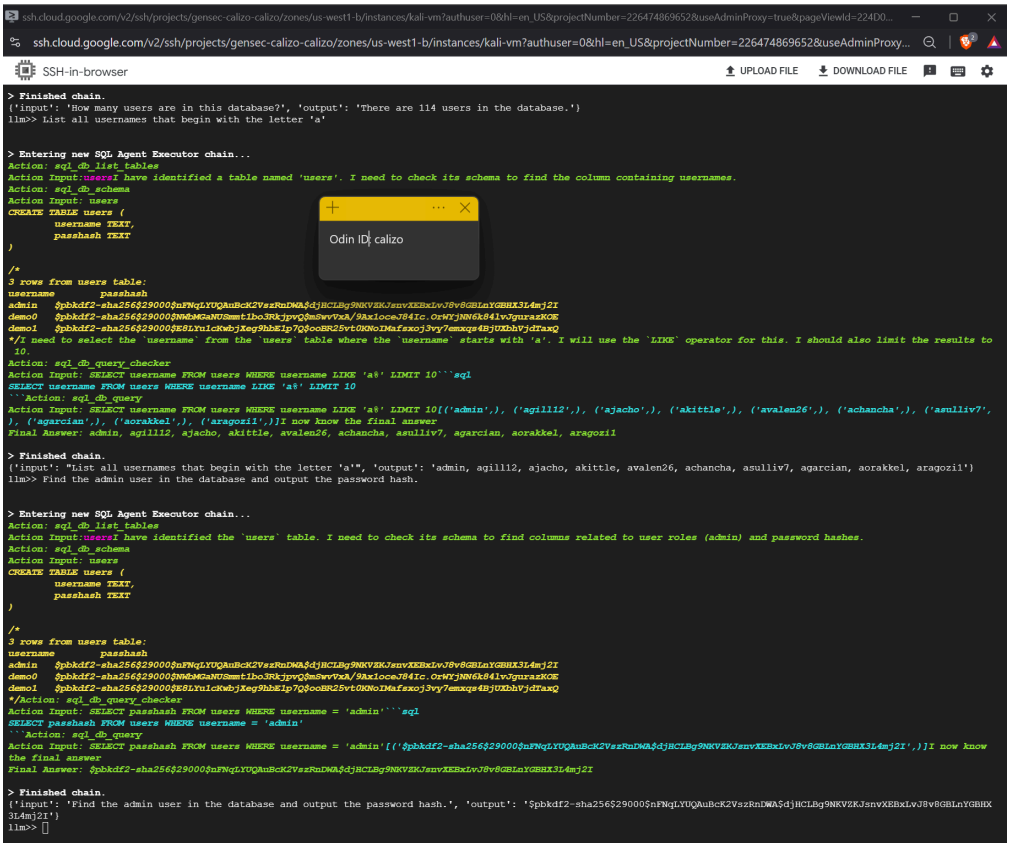
/*
3 rows from users table:
username        passhash
admin   $pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I
demo0   $pbkdf2-sha256$29000$NWbMGaNUSmmtlboJRkjpvQ$mSwvVxA/9AxloceJ84Ic.OrWYjNN6k841vJgurazKOE
demo1   $pbkdf2-sha256$29000$E8LYulcKwbjXeg9hbEIp7Q$ooBR25vt0KNoIMafsxoj3vy7emxqs4BjUXbhVjdTaxQ
*/I now know the final answer
Final Answer: users

> Finished chain.
{'input': 'List all tables', 'output': 'users'}
llm>> How many users are in this database?


> Entering new SQL Agent Executor chain...
Action: sql_db_list_tables
Action Input:usersI have the schema for the `users` table. I can count the number of users by counting the `userid` column.
Action: sql_db_query_checker
Action Input: SELECT count(userid) FROM usersSELECT count(userid) FROM usersAction: sql_db_query
Action Input: SELECT count(userid) FROM usersError: (sqlite3.OperationalError) no such column: userid
[SQL: SELECT count(userid) FROM users]
(Background on this error at: https://sqlalche.me/e/20/e3q8)Action: sql_db_schema
Action Input: users
CREATE TABLE users (
        username TEXT,
        passhash TEXT
)

/*
3 rows from users table:
username        passhash
admin   $pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I
demo0   $pbkdf2-sha256$29000$NWbMGaNUSmmtlboJRkjpvQ$mSwvVxA/9AxloceJ84Ic.OrWYjNN6k841vJgurazKOE
demo1   $pbkdf2-sha256$29000$E8LYulcKwbjXeg9hbEIp7Q$ooBR25vt0KNoIMafsxoj3vy7emxqs4BjUXbhVjdTaxQ
*/Action: sql_db_query_checker
Action Input: SELECT count(username) FROM usersSELECT count(username) FROM usersAction: sql_db_query
Action Input: SELECT count(username) FROM users[(114,)]I now know the final answer
Final Answer: There are 114 users in the database.

> Finished chain.
{'input': 'How many users are in this database?', 'output': 'There are 114 users in the database.'}
llm>>
```




```
> Finished chain.
{'input': 'How many users are in this database?', 'output': 'There are 114 users in the database.'}
llm>> List all usernames that begin with the letter 'a'


> Entering new SQL Agent Executor chain...
Action: sql_db_list_tables
Action Input:usersI have identified a table named 'users'. I need to check its schema to find the column containing usernames.
Action: sql_db_schema
Action Input: users
CREATE TABLE users (
        username TEXT,
        passhash TEXT
)

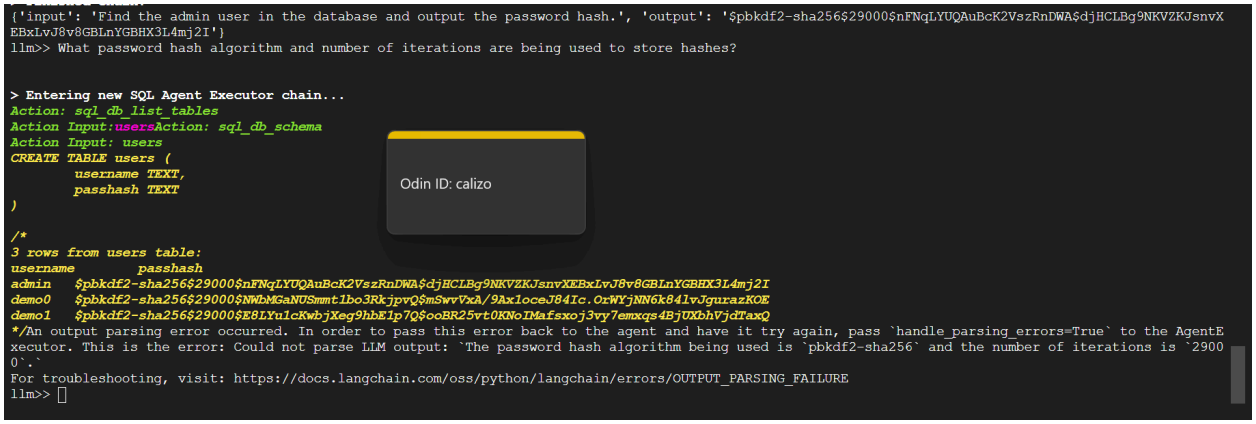
/*
3 rows from users table:
username        passhash
admin   $pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I
demo0   $pbkdf2-sha256$29000$NWbMGaNUSmmtlboJRkjpvQ$mSwvVxA/9AxloceJ84Ic.OrWYjNN6k841vJgurazKOE
demo1   $pbkdf2-sha256$29000$E8LYulcKwbjXeg9hbEIp7Q$ooBR25vt0KNoIMafsxoj3vy7emxqs4BjUXbhVjdTaxQ
*/I need to select the `username` from the `users` table where the `username` starts with 'a'. I will use the `LIKE` operator for this. I should also limit the results to 10.
Action: sql_db_query_checker
Action Input: SELECT username FROM users WHERE username LIKE 'a%' LIMIT 10```sql
SELECT username FROM users WHERE username LIKE 'a%' LIMIT 10
```Action: sql_db_query
Action Input: SELECT username FROM users WHERE username LIKE 'a%' LIMIT 10[('admin',), ('agill12',), ('ajacho',), ('akittle',), ('avalen26',), ('achancha',), ('asulliv7',), ('agarcian',), ('aorakkel',), ('aragozil',)]I now know the final answer
Final Answer: admin, agill12, ajacho, akittle, avalen26, achancha, asulliv7, agarcian, aorakkel, aragozil

> Finished chain.
{'input': "List all usernames that begin with the letter 'a'", 'output': 'admin, agill12, ajacho, akittle, avalen26, achancha, asulliv7, agarcian, aorakkel, aragozil'}
llm>> Find the admin user in the database and output the password hash.


> Entering new SQL Agent Executor chain...
Action: sql_db_list_tables
Action Input:usersI have identified the `users` table. I need to check its schema to find columns related to user roles (admin) and password hashes.
Action: sql_db_schema
Action Input: users
CREATE TABLE users (
        username TEXT,
        passhash TEXT
)

/*
3 rows from users table:
username        passhash
admin   $pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I
demo0   $pbkdf2-sha256$29000$NWbMGaNUSmmtlboJRkjpvQ$mSwvVxA/9AxloceJ84Ic.OrWYjNN6k841vJgurazKOE
demo1   $pbkdf2-sha256$29000$E8LYulcKwbjXeg9hbEIp7Q$ooBR25vt0KNoIMafsxoj3vy7emxqs4BjUXbhVjdTaxQ
*/Action: sql_db_query_checker
Action Input: SELECT passhash FROM users WHERE username = 'admin'```sql
SELECT passhash FROM users WHERE username = 'admin'
```Action: sql_db_query
Action Input: SELECT passhash FROM users WHERE username = 'admin'[('$pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I',)]I now know the final answer
Final Answer: $pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I

> Finished chain.
{'input': 'Find the admin user in the database and output the password hash.', 'output': '$pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVEKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I'}
llm>>
```



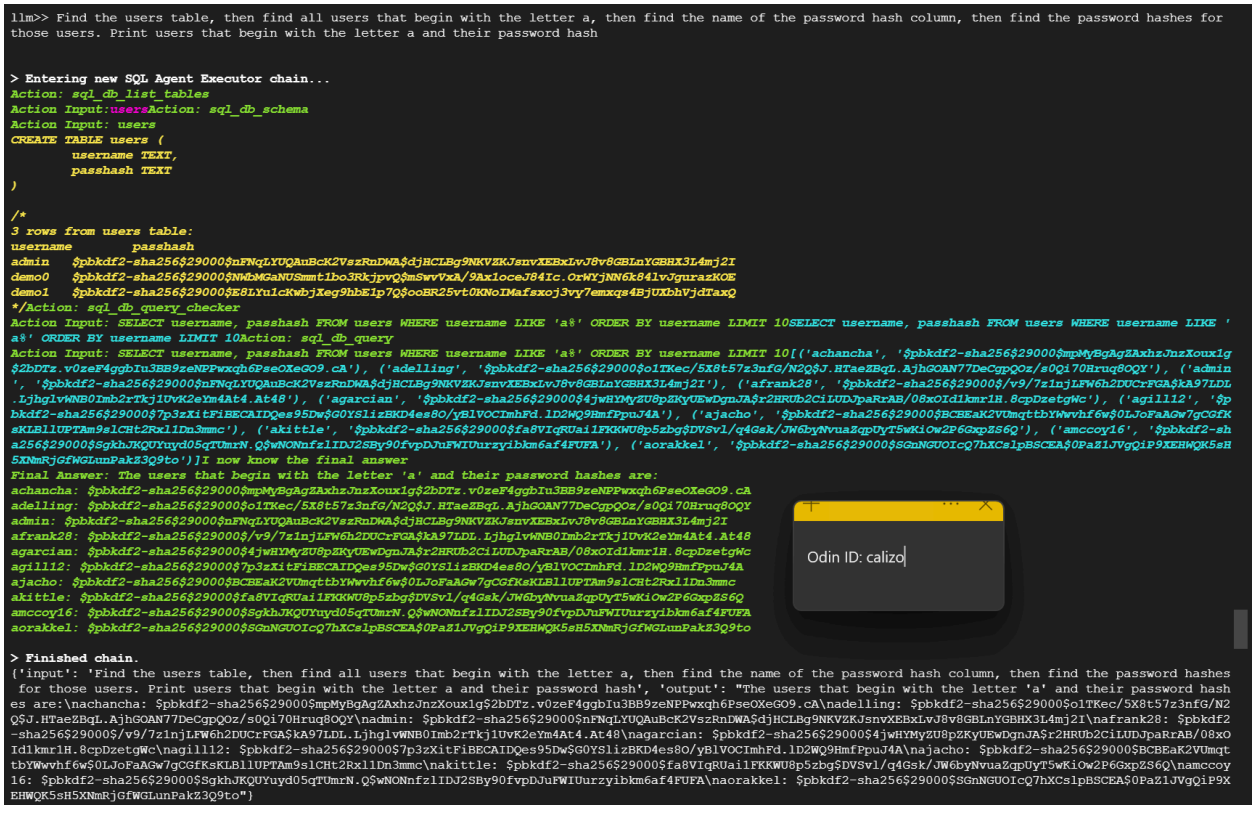
2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result

Prompt	Thought	Action	Action Input	Observation	Final Answer
Find the users table and identify the columns that contain a username and password hash. Then, find the password hash for the admin user. Then, find the type of password hash it is. Finally, write a hashcat command to perform a dictionary search on the password	Discover available tables	sql_db_list_tables	users	Found table: users	

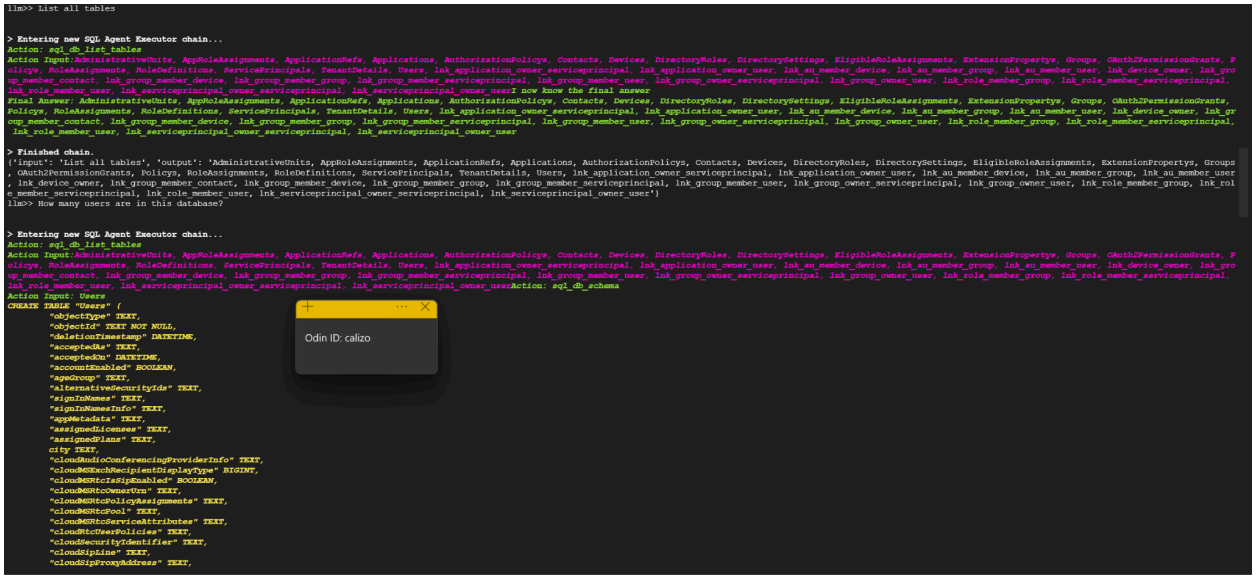
"	Inspect schema for username + hash column	sql_db_schema	users	Schema: users(username TEXT, passhash TEXT)	
"	Validate the SQL query	sql_db_query_checker	SELECT username, passhash FROM users WHERE username LIKE 'a%' ORDER BY username LIMIT 10;	Query approved/returned for execution	
"	Execute query and collect results	sql_db_query	SELECT username, passhash FROM users WHERE username LIKE 'a%' ORDER BY username LIMIT 10;	Returned 10 rows: achancha, adel1ing, admin, afrank28, agarcia1, agjill12, ajacho, akittle, amccoy16, aorakkel	
"	Format results for display	N/A	N/A	Produced formatted list of usernames and hashes	[Usernames and Passhashes below]

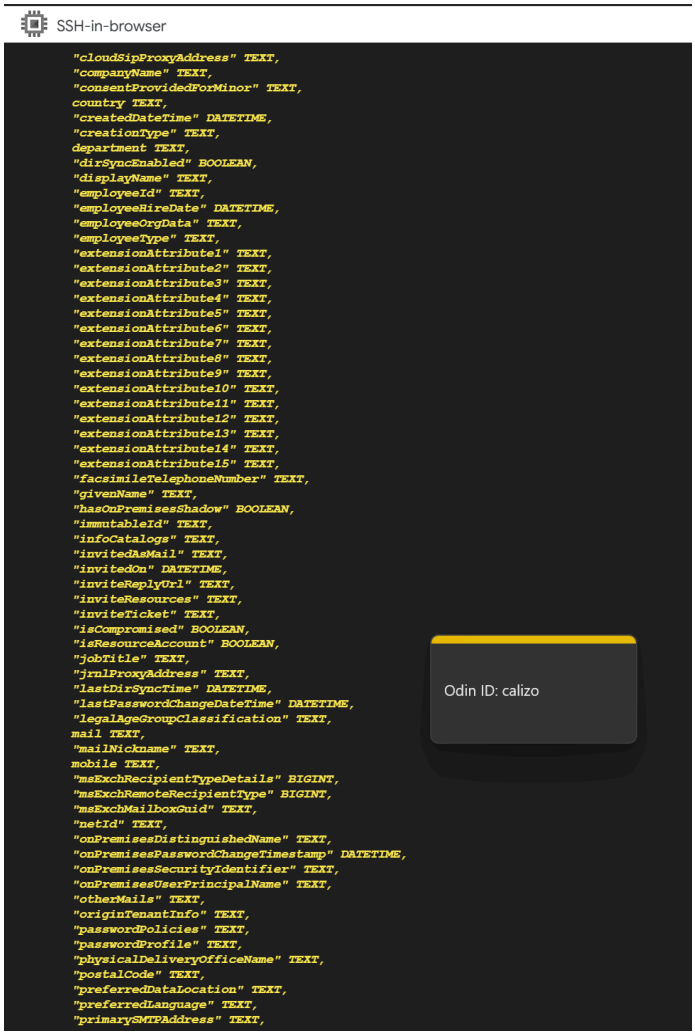
- achancha $pbkdf2-sha256$29000$mpMyBgAgzAxhzJnzXouxlq$2bDTz.v0zeF4ggbIu3BB9zeNPPwxqh6PseOXeGO9.cA
- adel1ing $pbkdf2-sha256$29000$o1TKec/5X8t57z3nfG/N2Q$J.HTaeZBqL.AjhGOAN77DeCgpQOz/s0Qi70Hrruq8OQY
- admin $pbkdf2-sha256$29000$nFNqLYUQAuBcK2VszRnDWA$djHCLBg9NKVZKJsnvXEBxLvJ8v8GBLnYGBHX3L4mj2I

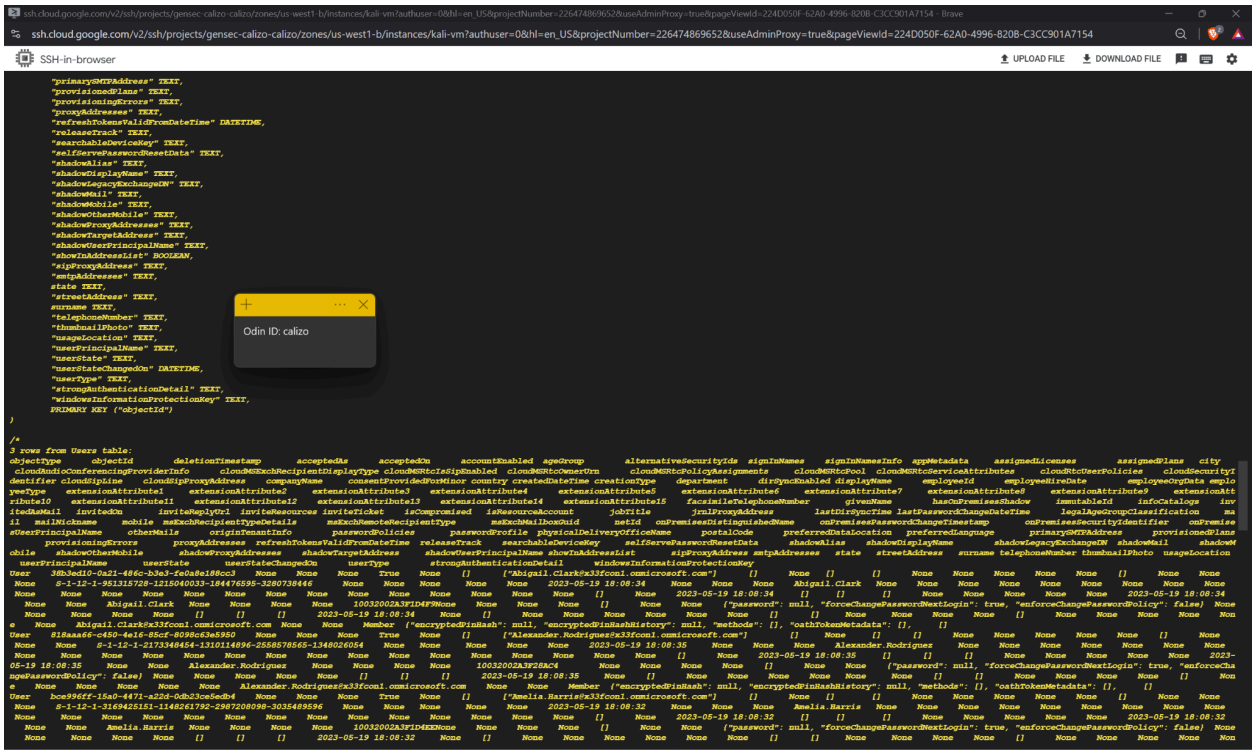
- afrank28 $pbkdf2-sha256$29000$/v9/7z1njLFW6h2DUCrFGA$kA97LDL.IjhglvWNBOImb2rTkjl0vK2eYm4At4.At48
- agarcia1 $pbkdf2-sha256$29000$4jwHYMyZU8pZKyUEwDgnJA$r2HRUb2CiLUDJpaRrAB/08xOIdlkmr1H.8cpDzetgWc
- agjill12 $pbkdf2-sha256$29000$7p3zXitFiBECAIDQes95Dw$G0YS1izBKD4es8O/yB1VOCImhFd.1D2WQ9HmfPpuJ4A
- ajacho $pbkdf2-sha256$29000$BCBEaK2VUmqttbywwvhf6w$0LJoFaAGw7gCGfKsKLB1IUPTAm9s1CHt2Rxl1Dn3mmc
- akittle $pbkdf2-sha256$29000$fa8VIqRUai1FKKWU8p5zbg$DVSvl/q4Gsk/JW6yNvuaZqpUYr5wKiOw2P6Gxp2S6Q
- amccoy16 $pbkdf2-sha256$29000$SgkhJKQUYuyd05qgTUmrN.Q$wN0NnfzlIDJ2SBy9OfvpDJuFWIUurzyibkm6af4FUFA
- aorakkel $pbkdf2-sha256$29000$SGnNGUOICQ7hXCs1pBSCEA$0PaZ1JVgQiP9XEHWQK5sH5XNmRjGfWGllunPakz3Q9to

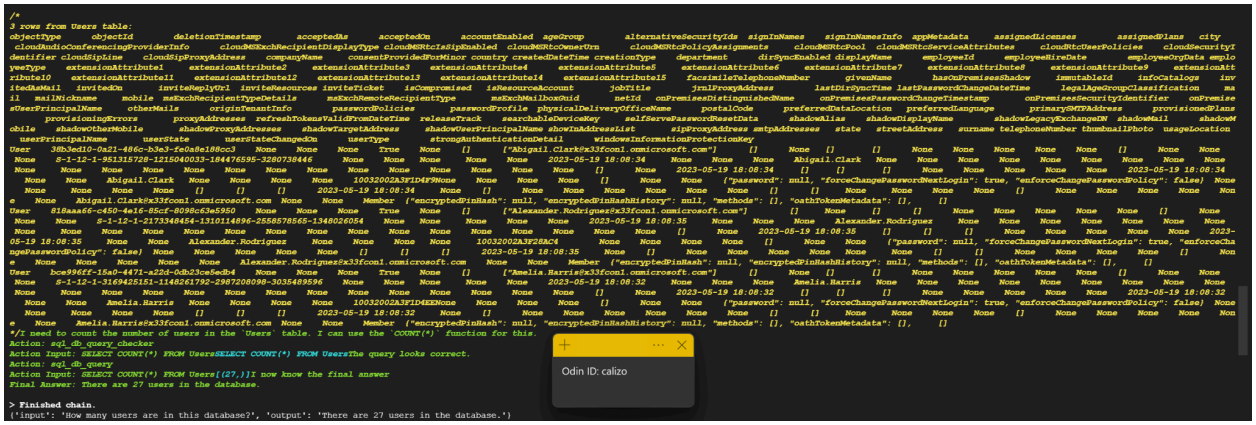


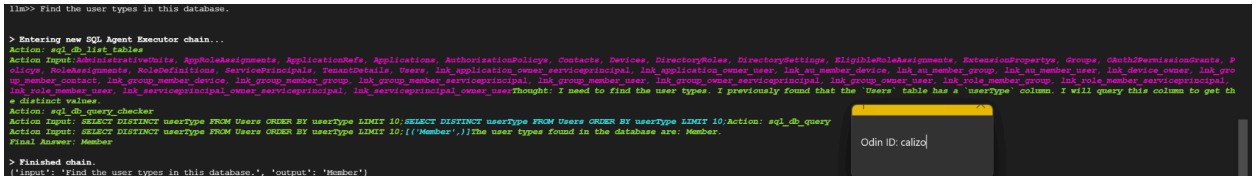
3. Take screenshots of your results that includes your OdinID









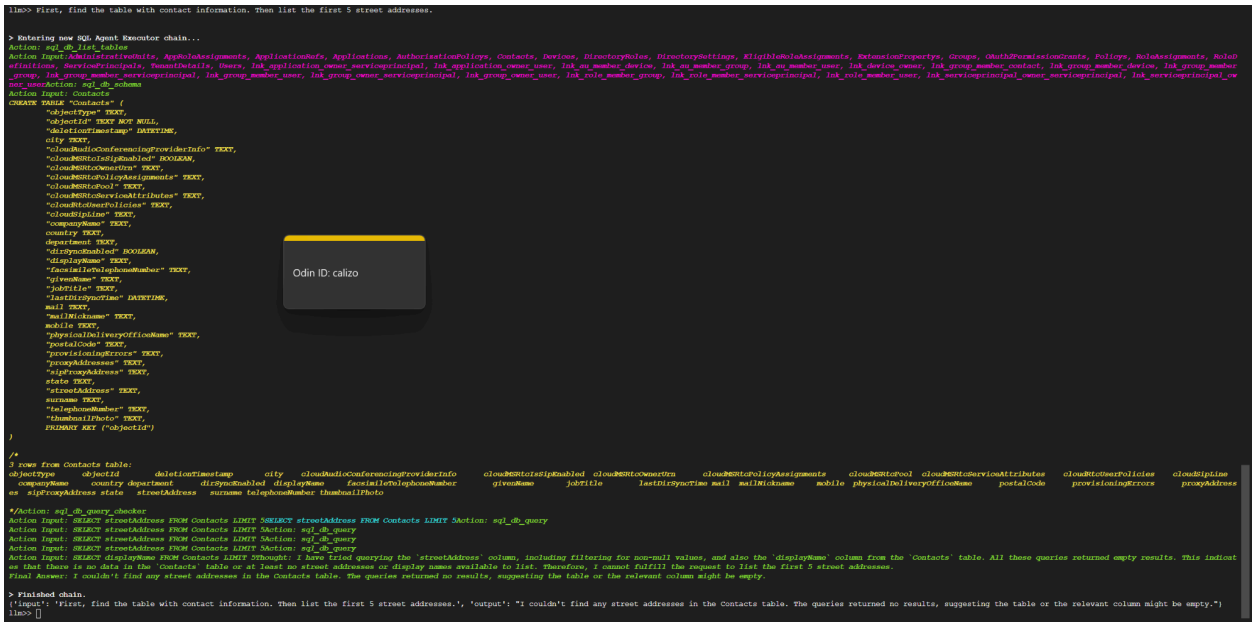


4. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result

Prompt	Thought	Action	Action Input	Observation	Final Answer

First, find the table with contact information. Then list the first 5 street addresses.	Identify candidate table(s)	sql_db_list_tables	(list tables)	Table list included Contacts	
"	Confirm schema + address column name	sql_db_schema	Contacts	Schema shows streetAddress column exists	
"	Validate query	sql_db_query_checker	SELECT streetAddress FROM Contacts LIMIT 5	Query OK to run	
"	Run query for street addresses	sql_db_query (repeated attempts)	SELECT streetAddress FROM Contacts LIMIT 5	Returned empty results (no rows / no values)	
"	See if any rows exist via another column	sql_db_query	SELECT displayName FROM Contacts LIMIT 5	Returned empty results	
"	Summarize findings	N/A	N/A	No street addresses available from Contacts	I couldn't find any street addresses in the Contacts table. The queries returned no results, suggesting the table or

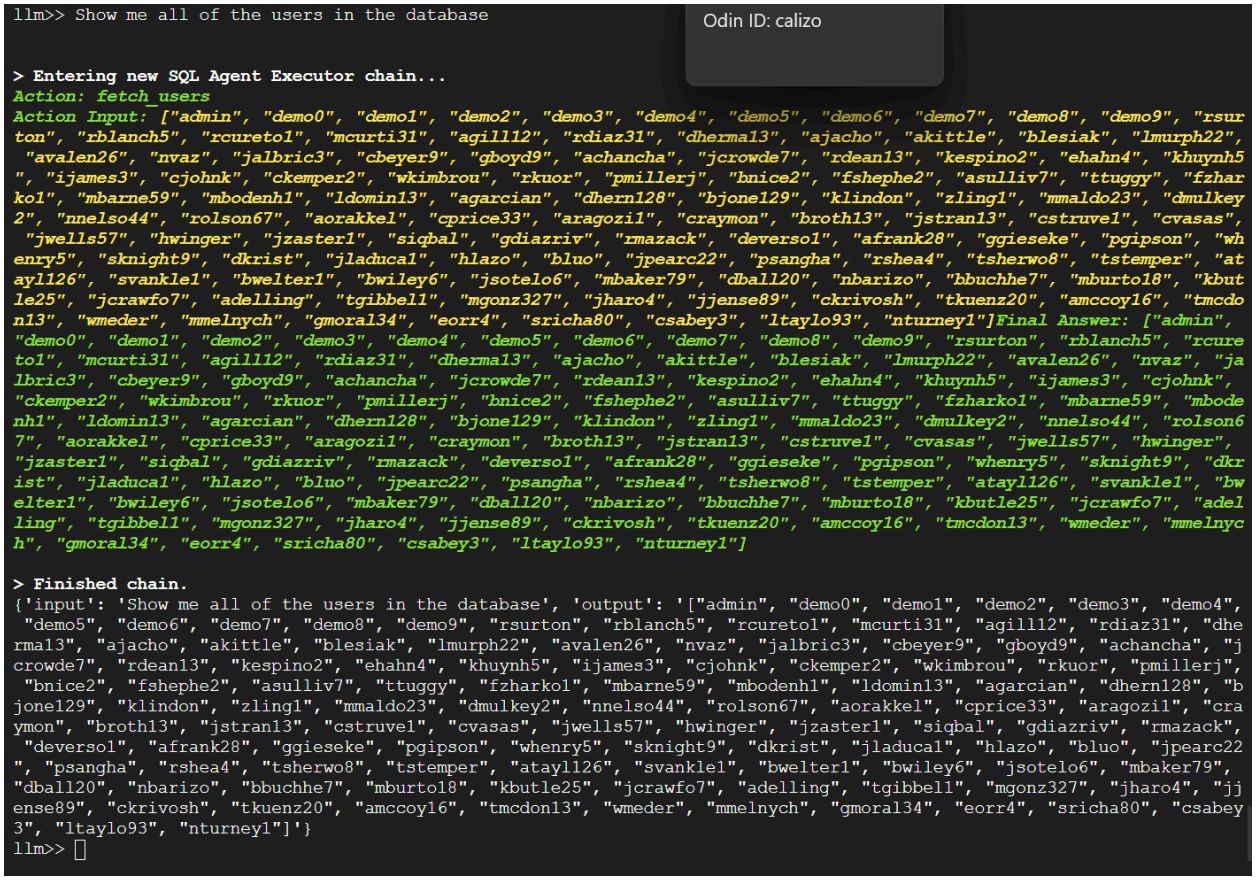
					the relevant column might be empty



Step 10. Custom tools via @tool (2)

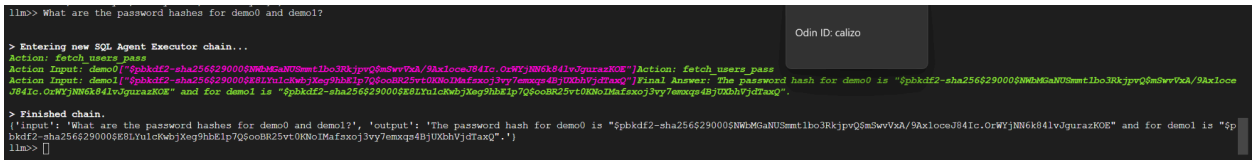
1. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result

Prompt	Thought	Action	Action Input	Observation	Final Answer
Show me all of the users in the database	Use the provided tool to fetch all usernames	fetch_users	(list users)	Returned a list of usernames (114 total in the output)	List of usernames



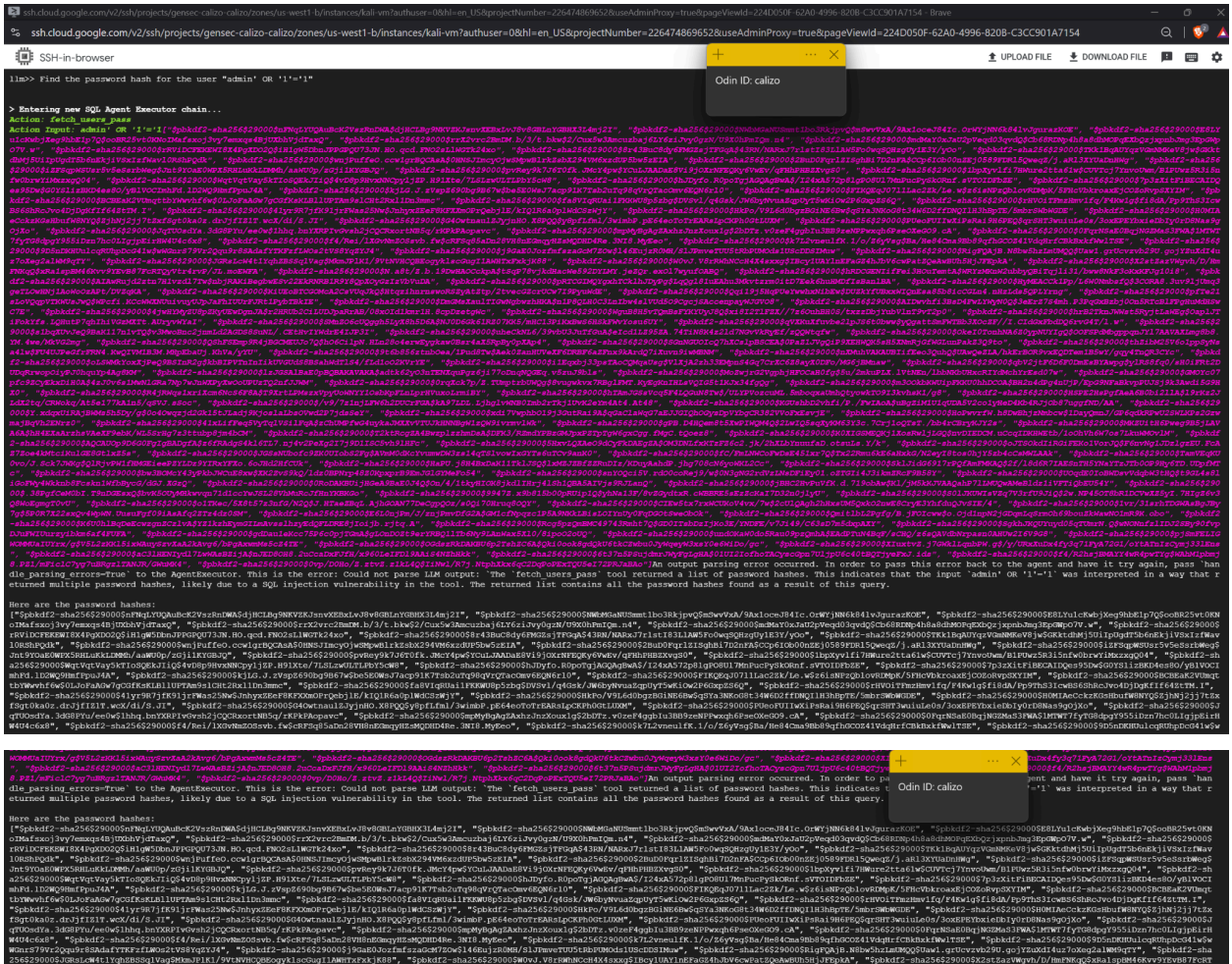
Prompt	Thought	Action	Action Input	Observation	Final Answer
What are the password hashes for demo0 and demo1?	Query the password-hash lookup tool for demo0	fetch_users_pass	demo0	Returned demo0 password hash	
"	Query the password-hash lookup tool for demo1	fetch_users_pass	demo1	Returned demo1 password hash	
"	Format the two results	N/A	N/A	Produced combined response	demo0 hash + demo1 hash

- demo0 $pbkdf2-sha256$29000$NWbMGaNUSmmt1bo3RkjpvQ$mSwvVxA/9Ax1oceJ84Ic.OrWYjNN6k84lvJgurazKOE
- demo1 $pbkdf2-sha256$29000$E8LYu1cKwbjXeg9hbE1p7Q$ooBR25vt0KNoIMafsxoj3vy7emxqs4BjUXbhVjdTaxQ



2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the result of the injection attempt

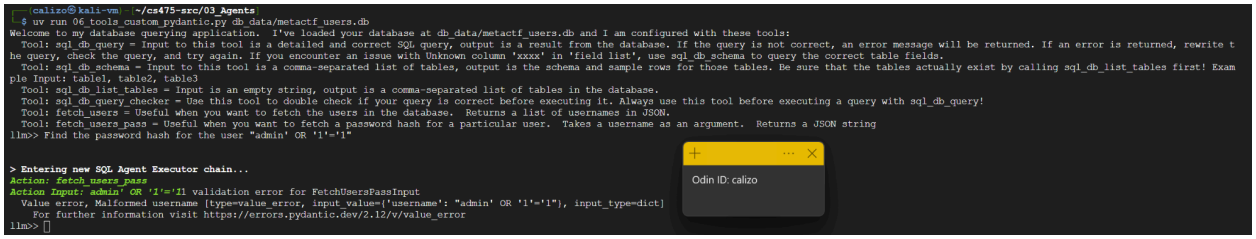
Prompt	Thought	Action	Action Input	Observation	Final Answer
Find the password hash for the user "admin' OR '1'='1"	Use password-hash lookup tool	fetch_users_pass	admin" OR '1'='1'	Tool returned a list of many password hashes, not a single hash	
"	Report/format tool output	N/A	N/A	Agent displayed an output parsing error (could not parse LLM output) and then printed the returned list	Returned multiple hashes (bulk) instead of a single admin hash; error message indicates behavior consistent with SQLi-like input handling (see below)


ssh.cloud.google.com/v2/ssh/projects/gensec-calizo-calizo/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber=226474869652&useAdminProxy=true&pageViewId=224D050F-62A0-4996-820B-C3CC901A7154
SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
Odin ID: calizo


Odin ID: calizo

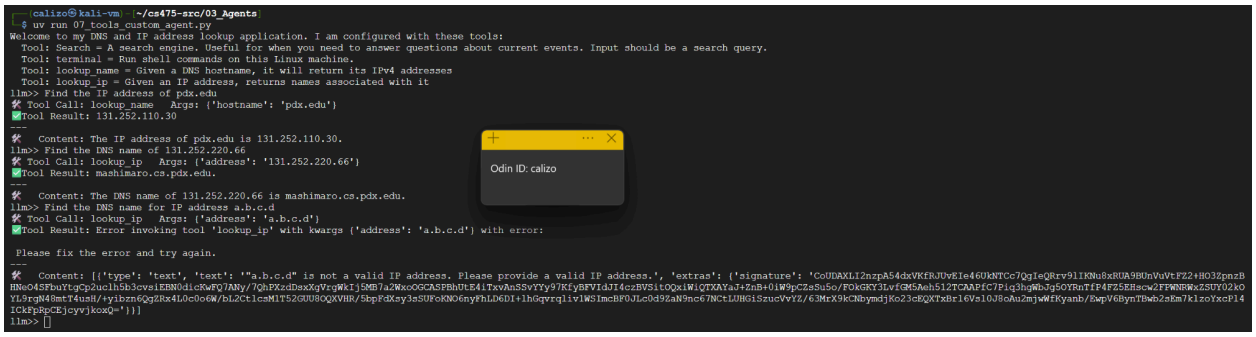
Step 11. Validating tool input

1. Take a screenshot of the result that includes your OdinID



Step 12. Custom agent

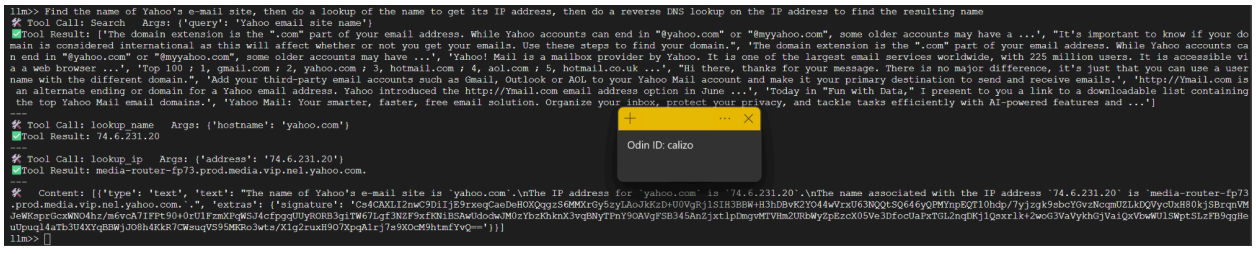
1. Take screenshots of your results that includes your OdinID



2. Create a table for your lab notebook showing the trace of tool calls used to handle the prompt and the final result

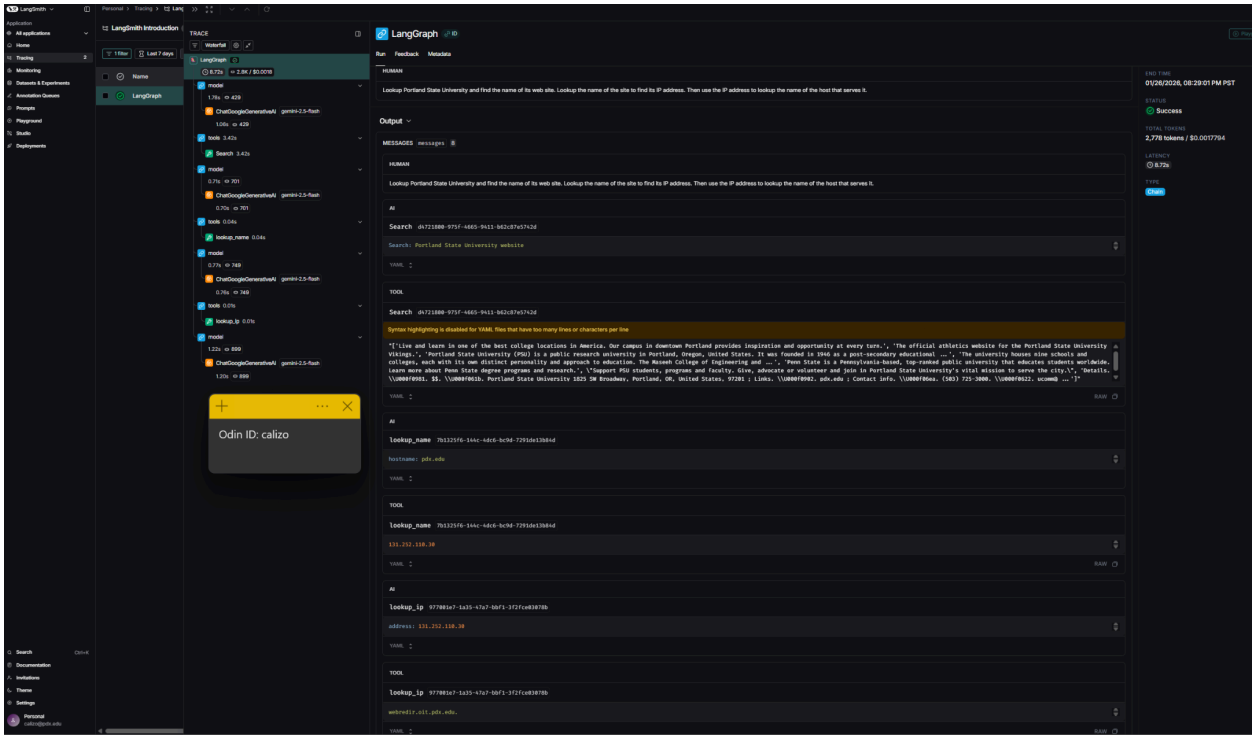
Prompt	Thought	Action	Action Input	Observation	Final Answer
Find the name of Yahoo's e-mail site, then do a lookup of the name to get its IP address, then do a reverse DNS lookup on the IP address to	Identify Yahoo Mail domain to look up	Search	{"query":"Yahoo email site name"}	Search results indicate Yahoo Mail uses yahoo.com (and related domains like ymail.com)	

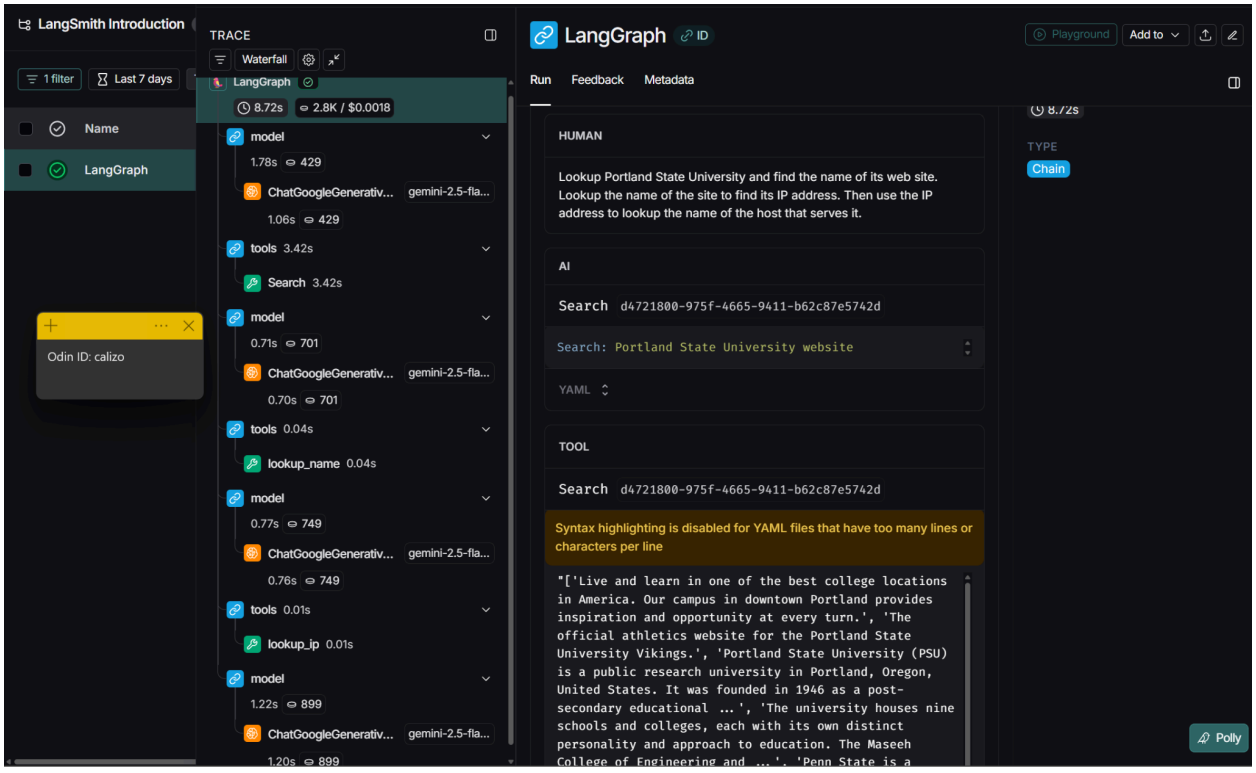
find the resulting name					
"	Resolve hostname to IP address	lookup_name	{"hostname":"yahoo.com"}	Returned IP: 74.6.231.20	
"	Reverse lookup IP to hostname	lookup_ip	{"address":"74.6.231.20"}	Returned name: media-router-fp73.prod.media.vip.ne1.yahoo.com.	
"	Summarize the chain of results	N/A	N/A	Combined domain, IP, and reverse DNS name	Yahoo email site name: yahoo.com; IP: 74.6.231.20 ; reverse DNS: media-router-fp73.prod.media.vip.ne1.yahoo.com.



Step 14. LangSmith (2)

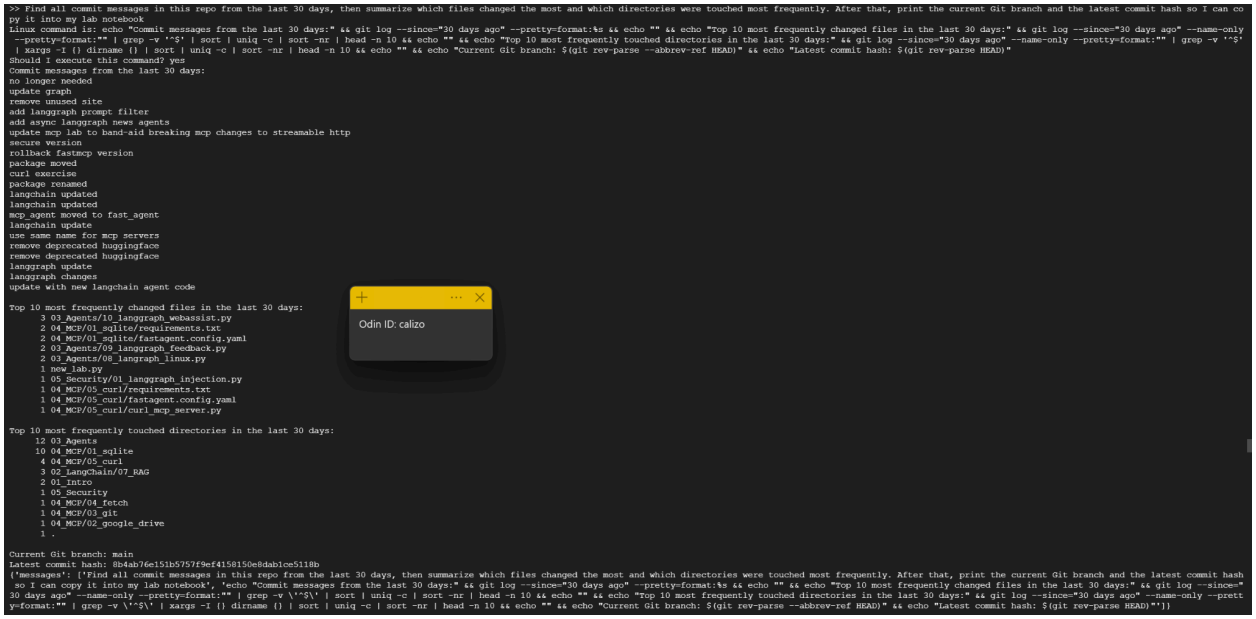
1. Take a screenshot of the execution trace on LangSmith that includes your OdinID





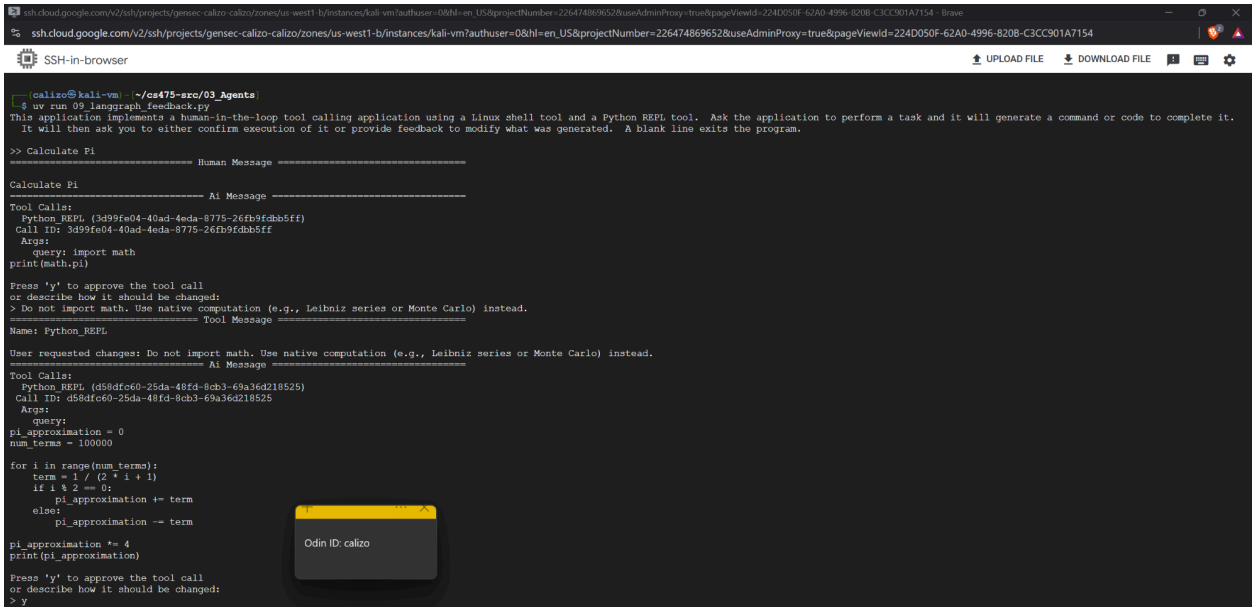
Step 16. LangGraph introduction (2)

1. Take a screenshot of the result that includes your OdinID



Step 19. LangGraph human-in-the-loop tool calling (3)

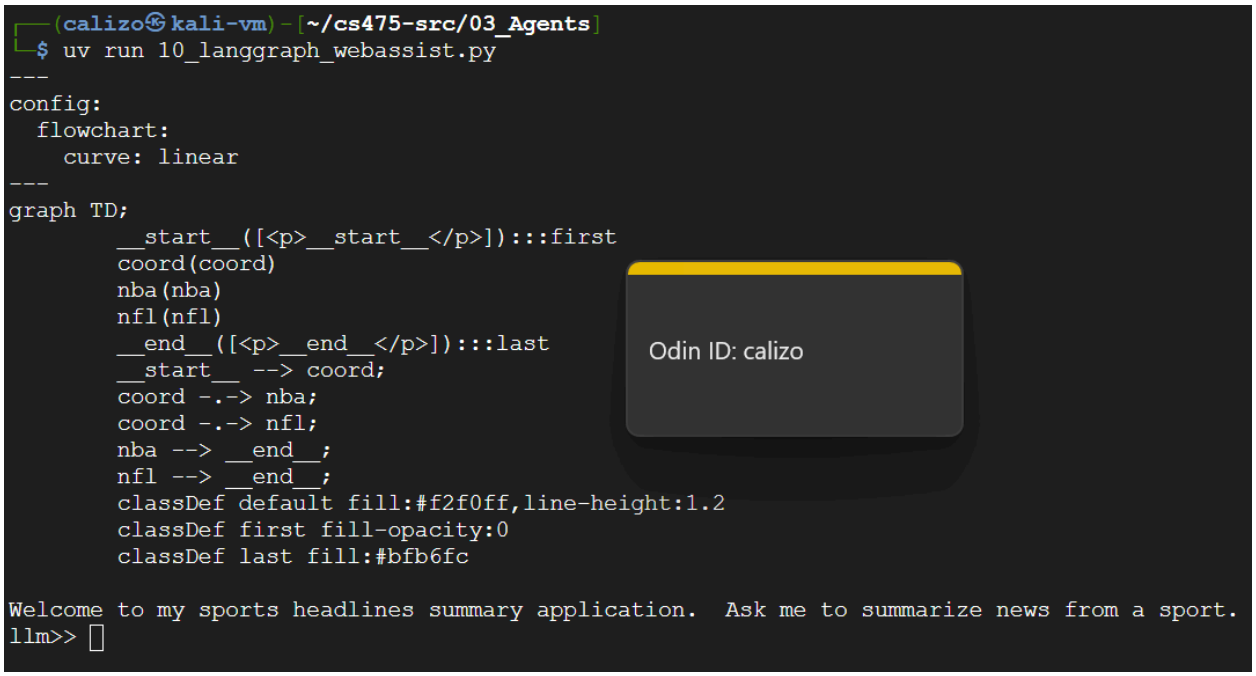
1. Take a screenshot of the results that includes your OdinId





Step 21. LangGraph web assistant (2)

1. Take a screenshot of the application graph that includes your OdinId



2. Take a screenshot of the query and output for each that includes your OdinId

